

模型微调手册

写在前面

模型微调需要借助GPU加速或混合精度等进行训练加速和优化，大量代码并不涉及直接的微调，为了将我们的主要精力集中在微调核心环节，我们推荐使用模型多卡训练框架[trainlab](#)，依赖该框架只需要简单配置和少量代码即可实现主流transformers风格的模型进行微调。

现如今深度学习模型遵循预训练+微调的范式，在大量无标注的数据中进行无监督训练，让模型学习到该数据领域的通用表示，具备了强大的知识和泛化能力，在预训练模型的基础上使用垂域数据（如针对金融问答任务，垂域数据是金融领域的问答对）进行微调。微调的方式有很多种，如全量微调和参数高效微调（PEFT），全量微调对数据量和设备要求较高，而PEFT仅调整模型中的特定几个层，在大幅减少显存、数据需求的同时在表现上达到全量微调的效果。PEFT中又以Lora最为热门。

本手册以trainlab为训练框架，实现在Chinses-CLIP图文检索预训练模型的基础上实现Lora微调。

TrainLab框架简介

trainlab框架参考OpenMMLab采用注册模式（Registry Pattern）组织代码，整个框架将深度学习训练过程抽象为三大模块：

- MODELS: 注册所有深度学习模型
- DATASETS: 注册所有数据集
- TRAINERS: 注册所有训练方法

注册机制示例（自定义数据集，其他模块方法相同，注意所有注册文件都需要在train.py中import注册机制才能成功运行）：

代码块

```
1  import os
2  from .dataset_base import DatasetBase
3  from trainlab.builder import DATASETS # DATASETS注册表
4
5  @DATASETS.register_module() # 调用注册函数，将StackOverflowDataset这个类注册到
   DATASETS注册表中
6  class StackOverflowDataset(DatasetBase):
7      name = 'train-sample'
8
9      def __init__(self, root, split='train', transform=None,
   cache_in_memory=False):
10         super().__init__(transform, cache_in_memory)
11
```

```

12         self.data_dir = os.path.join(root, self.name, split)
13         # self.data = self.load_data(self.data_dir).select(range(100))
14         # 在trainlab框架中，DatasetBase继承了torch.Dataset，重写了
        __getitem__(self, index)，只需将数据整理好赋值给self.data即可
15         self.data = self.load_data(self.data_dir)
16
17     def load_data(self, path):
18         if not os.path.exists(path):
19             raise FileNotFoundError(f"Dataset path not found: {path}")
20
21         dataset = load_from_disk(path)
22         return dataset
23
24     def collate_fn(self, batch):
25         """ Instructs how the DataLoader should process the data into a
        batch"""
26
27         text = [item['text'] for item in batch]
28         tabular = torch.stack([torch.tensor(item['tabular']) for item in
        batch])
29         labels = torch.stack([torch.tensor(item['label']) for item in batch])
30
31         return {'text': text, 'tabular': tabular, 'label': labels}
32

```

对于训练代码，在trainlab框架下我们仅需继承BaseTrainer然后只需要重写以下两个函数即可：

- `def custom_setup(self, rank, world_size):` 由于DDP下的多卡训练需要初始化在每个进程中初始化一个模型，如果要对模型进行修改（如增加lora层）即在这个函数中定义。
- `def run_one_epoch(self, epoch, total_epoch, data_loader, rank, device, train=True):` 训练每个epoch的逻辑。

对于支持的模型，trainlab支持自定义模型以及transformers风格的所有主流模型。transformers风格的模型支持请参考[文件](#)。

trainlab目录结构如图所示：

 config	配置文件
 datasets	数据集Dataset
 model	模型文件
 module	功能模块文件
 trainers	训练的核心代码
 utils	工具文件
 __init__.py	
 base_trainer.py	trainers中的文件的父类
 builder.py	注册机制的核心文件

trainlab的基础使用流程

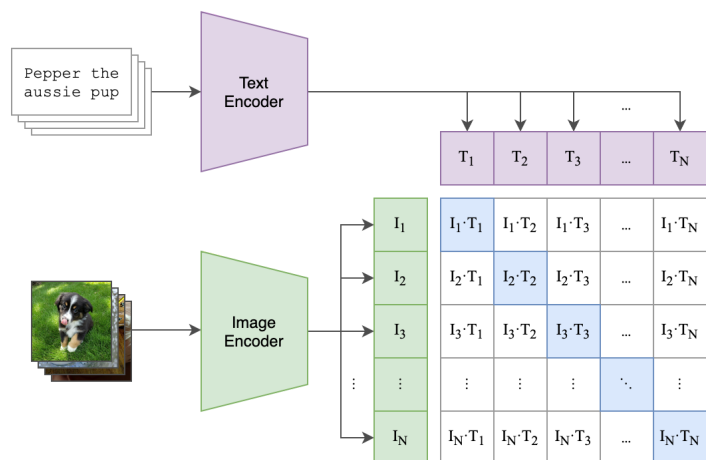
代码块

```
1  # 创建conda环境
2  conda create --name trainlab python=3.10
3  conda activate trainlab
4  pip install -r requirements.txt
5
6  # clone项目
7  git clone https://github.com/Kivw/trainLab.git
8  cd trainlab
9
10 # 修改好配置文件然后执行, 'bash scripts cuda_visible_device config_file'
11 bash scripts/peft_chinese_clip.sh 1,2,3
    /data/lj/task/trainlab/trainlab/config/peft_chinese_clip.yaml
```

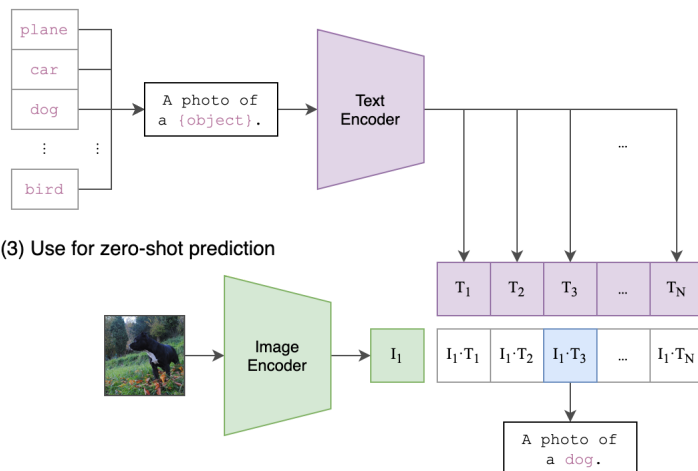
一、数据集标注和Dataset准备

Chinese-CLIP 是 CLIP 模型的中文版本，采用了与原始 CLIP 相似的对比学习框架。Chinese-CLIP 的结构由 **视觉编码器（ViT/ResNet）** 提取图像向量、**文本编码器（中文预训练语言模型）** 提取文本向量，并通过 **对比学习** 在向量空间中对齐图文特征。

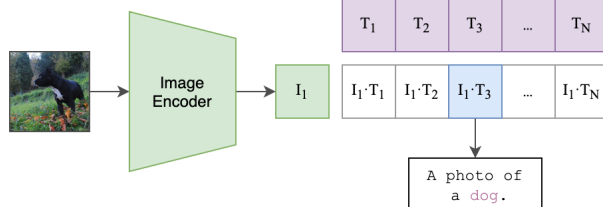
(1) Contrastive pre-training



(2) Create dataset classifier from label text



(3) Use for zero-shot prediction



Chinese-CLIP所需要的训练数据为图片数据对(image,text)。我们采用开源标注工具Label Studio进行数据标注。

 **Label Studio** 是一款开源、多功能的数据标注工具，支持对文本、图像、音频、视频、时间序列等多模态数据进行标注。

特点包括：

- **多模态支持**：图像、视频、文本、音频、HTML、时间序列等。
- **灵活配置**：通过 XML/JSON 配置标注界面和任务类型。
- **易于集成**：支持 Python SDK、REST API，可与深度学习训练管道结合。
- **团队协作**：可部署服务器端进行多用户协作标注。
- **扩展性**：支持自定义标注组件和自动化标注。

Label studio的安装和使用

代码块

```
1 # 使用 pip 安装
2 pip install label-studio
3 # 启动
4 label-studio start
```

启动完成后，会在浏览器中打开一个操作页面。label studio具备极高的标注自由度，可以依据项目需求自定义标注格式和方式，具体教程可参考[label studio中文教程](#)。针对Chinese_CLIP这个任务，我们使用了如下的标注配置：

代码块

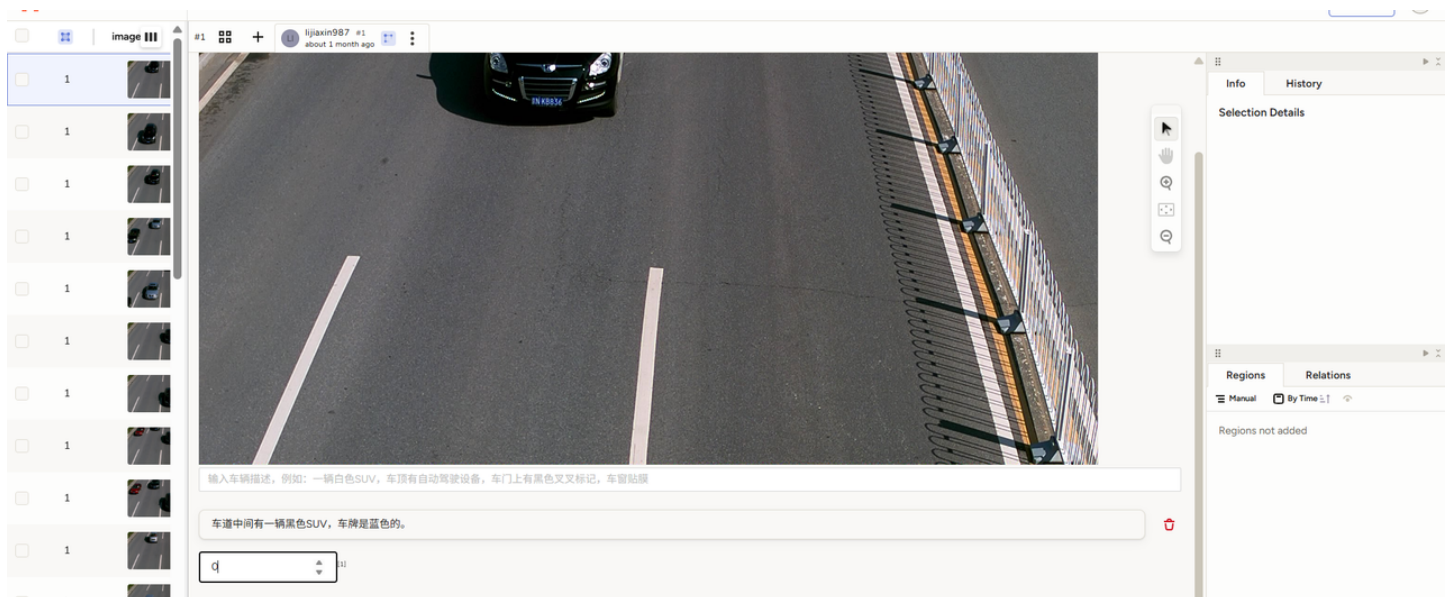
```
1 <View>
2   <Image name="image" value="$image"/>
```

```

3
4     <!-- 文本描述 -->
5     <TextArea name="description" toName="image"
6         placeholder="输入车辆描述，例如：一辆白色SUV，车顶有自动驾驶设备，车门上有
        黑色叉叉标记，车窗贴膜"/>
7
8     <!-- 类别标签（整数型） -->
9     <Number name="category" toName="image" valueType="integer"
10         minValue="0" maxValue="10"
11         placeholder="输入该图片的类别编号，例如：0表示SUV，1表示轿车"/>
12 </View>
13

```

可视化为：



完成数据标注后，我们得到的标注数据大概是这样的：

```

代码块
1  [
2    {
3      "image": "images\\0.jpg",
4      "id": 0,
5      "class": "车顶部有额外设备的车"
6    },
7    {
8      "image": "images\\1.jpg",
9      "id": 0,
10     "class": "车顶部有额外设备的车"
11   },
12   {
13     "image": "images\\2.jpg",

```

```

14     "id": 0,
15     "class": "车顶上有额外设备的车"
16 }
17 ]

```

注意，直接从label studi导出的数据子啊image这一栏的文件路径是相对于label studio的，我们需要自己写一个脚本修改这个路径为相对数据集根目录的路径。

至此数据标注的工作结束，我们需要依据我们的任务加载数据，即写我们自己的torch.Dataset。

torch.Dataset

我们以uft101图文对数据集为例分享trainlab的使用和Chinese-CLIP微调数据集Dataset的构建过程。

代码块

```

1  import os
2  import pickle
3  import json
4  from trainlab.builder import DATASETS
5  from .dataset_base import DatasetBase, Datum
6
7  @DATASETS.register_module()
8  class UCF101Dataset(DatasetBase):
9
10     dataset_dir = "ucf101"
11
12     def __init__(self, root, split='train', transform=None,
13 cache_in_memory=False):
14         """
15         args:
16             root: 数据集根目录;
17             split:数据集类型;
18             transform: 对数据进行变化增强;
19             cache_in_memory: 是否使用缓存数据
20         """
21         root = os.path.abspath(os.path.expanduser(root))
22         self.dataset_dir = os.path.join(root, self.dataset_dir) # 数据集目录
23         self.image_dir = os.path.join(self.dataset_dir, "UCF-101-midframes") #
24         images目录
25         self.split_path = os.path.join(self.dataset_dir,
26 "split_zhou_UCF101_chinese.json") # label文件路径
27         self.split_using_data = os.path.join(self.dataset_dir,
28 "split_using_data") # 存放划分好的数据集
29         if not os.path.exists(self.split_using_data):
30             os.makedirs(self.split_using_data)
31         super().__init__(transform, cache_in_memory)

```

```

28
29     # 将数据及其路径从json文件中读取出来
30     train, val, test = self.read_split(self.split_path, self.image_dir) #
    将数据组织成我们需要的形式
31
32     # 预处理样本数据
33     preprocessed = os.path.join(self.split_using_data, f"using_data.pkl")
34
35     if os.path.exists(preprocessed):
36         print(f"Loading preprocessed few-shot data from {preprocessed}")
37         with open(preprocessed, "rb") as file:
38             data = pickle.load(file)
39             train, val = data["train"], data["val"]
40     else:
41
42         data = {"train": train, "val": val}
43         print(f"Saving preprocessed few-shot data to {preprocessed}")
44         with open(preprocessed, "wb") as file:
45             pickle.dump(data, file, protocol=pickle.HIGHEST_PROTOCOL)
46
47     # 对self.data进行赋值, DatasetBase已经实现和了__item__(self, index)
48     if split == 'train':
49         self.data = train
50     elif split == 'val':
51         self.data = val
52     elif split == 'test':
53         self.data = test
54     else:
55         raise ValueError(f"Invalid split value: {split}")
56     # 组织数据格式
57     def read_split(self, split_path, path_prefix):
58         def _convert(item_list):# 将相对路径转换为绝对路径
59             out = []
60             for impath, label, classname in item_list:
61                 impath = os.path.join(path_prefix, impath)
62                 item = (impath, label, classname)
63                 out.append(item)
64
65             return out
66
67         with open(split_path, "r") as f:
68             data = json.load(f)
69             train = _convert(data["train"])
70             val = _convert(data["val"])
71             test = _convert(data["test"])
72             return train, val, test
73

```

二、使用PEFT库对Chinese_clip进行lora微调训练代码

Chinese-clip预训练模型下载

代码块

```
1 pip install modelscope # 安装modelscope
2 modelscope download --model AI-ModelScope/chinese-clip-vit-base-patch16 --
  local_dir ./model # 下载预训练权重到model文件夹
```

对Chinese-clip增加lora层

PEFT是transformers系列工具中高效参数微调的库，仅需要通过几步简单的配置就可以实现网模型中插入lora层。

接下来我们以[文件](#)来解释如何利用trainlab和peft库写微调的核心训练代码。

代码块

```
1 import os
2 import torch
3 import torch.nn as nn
4 from peft import get_peft_model, LoraConfig, TaskType
5 from transformers import ChineseCLIPProcessor
6 from trainlab.builder import TRAINERS
7 from trainlab.base_trainer import BaseTrainer
8 from trainlab.utils.tools import cls_acc
9 from PIL import Image
10
11 @TRAINERS.register_module()
12 class PEFTCLIPTrainer(BaseTrainer):
13     def __init__(self,
14                 model, # 实例化后的CLIP模型
15                 model_name_or_path, # clip预训练模型的权重文件夹
16                 log_queue, # log_queue用于多进程见log通信
17                 project_name, # 项目名字, 用于打印log
18                 loss_fn=None, # 损失函数, 有配置文件给出
19                 epochs=1,
20                 optimizer_class=None, # 优化器, 在BaseTrainer中实例化优化器
21                 optimizer_kwargs=None, # 优化器参数
22                 scheduler_class=None, # 调度器
23                 scheduler_kwargs=None, # 调度器参数
```



```

24         save = True,
25         output_dir = './',
26         output_filename='weight'):
27     super(PEFTCLIPTrainer, self).__init__(model, log_queue,
project_name,loss_fn, epochs, optimizer_class, optimizer_kwargs,
scheduler_class, scheduler_kwargs,save,output_dir, output_filename)
28
29     # lora微调的配置文件
30     self.peft_text_config = LoraConfig(
31         # task_type=TaskType.FEATURE_EXTRACTION,
32         inference_mode=False,
33         r=8, # lora中低秩矩阵的秩
34         lora_alpha=16,
35         lora_dropout=0.1,
36         target_modules=['query','value','q_proj','v_proj'] # 将被替换成lora层
的名称，实践经验表明微调这两个模块效果最好
37     )
38
39     self.processor =
ChineseCLIPProcessor.from_pretrained(model_name_or_path)
40     self.instruct = '这张图片描述的内容是'
41
42     def custom_setup(self, rank, world_size):
43         '''再多进程情况下，每个进程中的模型是一个单独的实例，那么对模型进行操作时要在子进
程中进行，重载这个函数即可插入特定的初始化逻辑再子进程中'''
44         # transformers风格的模型被包装在warpper里面，但是没有暴露model的属性，而是用
origin_model承接
45         # add LoRA layers to text model
46         self.model.origin_model = get_peft_model(self.model.origin_model,
self.peft_text_config)
47         # add LoRA layers to vision model
48         # self.model.origin_model.vision_model =
get_peft_model(self.model.origin_model.vision_model, self.peft_vision_config)
49         # count the number of trainable parameters
50         if rank == 0:
51             self.model.origin_model.print_trainable_parameters()
52
53
54     def run_one_epoch(self, epoch,totoal_epcoh, data_loader, rank, device,
train=True):
55
56         self.model.train() if train else self.model.eval()
57
58         # initialize variables
59         loss_accumulated = 0
60         acc_train = 0
61         tot_samples = 0

```

```

62
63
64     # 只在主进程rank=1中打印日志
65     data_loader_iter = self.wrap_data_loader_with_tqdm(
66         rank=rank,
67         data_loader=data_loader,
68         epoch=epoch,
69         train_model=train
70     )
71
72     for idx, (impath, labels, classname) in enumerate(data_loader_iter):
73
74         images = [Image.open(path).convert('RGB') for path in impath]
75         texts = [self.instruct + c for c in classname]
76
77         inputs = self.processor(text=texts, images=images,
return_tensors="pt", padding=True).to(device)
78         output = self.model(**inputs, return_loss=True)
79         loss = output.loss
80         logits_per_image = output.logits_per_image
81         labels = torch.arange(logits_per_image.shape[0]).to(device)
82         acc_train += cls_acc(logits_per_image, labels) * labels.shape[0]
83         loss_accumulated += loss.item() * labels.shape[0]
84         tot_samples += labels.shape[0]
85
86         if train:
87             self.optimizer.zero_grad()
88             loss.backward() # 多卡中, 会自动同步多gpu梯度
89             self.optimizer.step()
90             self.scheduler.step()
91
92     # cpu等待当前GPU上所有计算完成, 每个gpu进程都调用这个函数, 则cpu就需要等待所
有gpu完成梯度同步
93     torch.cuda.synchronize(device)
94
95     if rank == 0:
96         acc_train /= tot_samples
97         loss_accumulated /= tot_samples
98         current_lr = self.scheduler.get_last_lr()[0]
99
100     # print metrics to console
101     self.logger.info(
102         f"epoch={epoch},\
103         loss={round(loss_accumulated, 3)},\
104         acc_train={round(acc_train, 3)},\
105         lr={round(current_lr, 6)}"
106     )

```

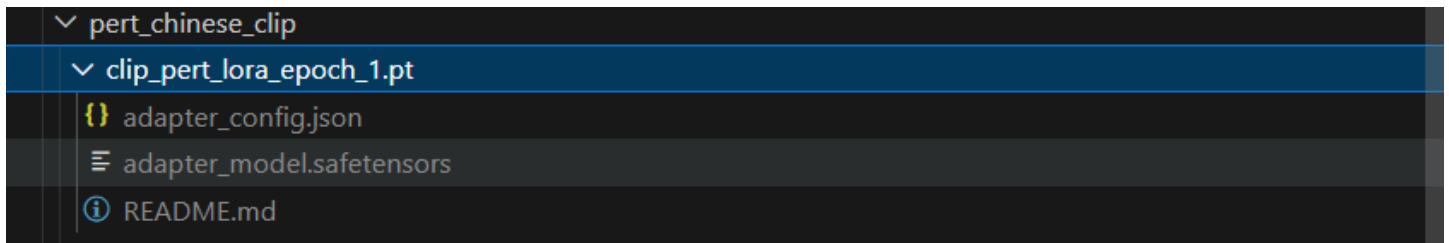
```

107
108         self.save_best_model(epoch, loss_accumulated, train=train) # 仅在 eval
模式下保存最优模型
109
110
111     def save_best_model(self, epoch, avg_loss_epoch, train=False):
112         """
113         保存当前最优模型权重（仅在 eval 模式且损失更好时保存）
114
115         Args:
116             epoch (int): 当前训练轮数
117             avg_loss_epoch (float): 当前轮平均损失
118             train (bool): 是否为训练阶段（True 表示训练阶段，不保存）
119         """
120         if not self.save or train:
121             return
122
123         # 只有当当前轮损失更优时才保存
124         if avg_loss_epoch >= self.eval_loss:
125             return
126
127         # 增加项目目录
128         dir_path = os.path.join(self.output_dir, self.project_name)
129         # 创建输出文件夹
130         os.makedirs(dir_path, exist_ok=True)
131
132         # 新模型保存路径
133         self.file_path = os.path.join(
134             os.path.abspath(dir_path),
135             f"{self.output_filename}_epoch_{epoch}.pt"
136         )
137
138         # 删除上一个最优模型
139         if hasattr(self, 'epoch_best_model') and self.epoch_best_model is not
None:
140             file_path_prev = os.path.join(
141                 os.path.abspath(dir_path),
142                 f"{self.output_filename}_epoch_{self.epoch_best_model}.pt"
143             )
144             if os.path.exists(file_path_prev):
145                 os.remove(file_path_prev)
146
147         # 使用 PEFT 方法保存模型，只保存增量参数，用DDP包装模型后会模型结构会增加module
这个字段
148         self.model.module.origin_model.save_pretrained(self.file_path)
149
150

```

三、模型输出和调用

Lora微调时保存的权重文件只有增量权重，即不包含预训练权重，所以加载方法有所不同。我们先来看以下保存文件的内容：



Lora模型保存的文件包含三部分

- .json： 配置文件
- .safetensors: 权重文件
- readme.md

模型调用与权重合并

代码块

```
1  from peft import PeftModel, PeftConfig
2
3  peft_model_id = f"
    {model_name_or_path}_{peft_config.peft_type}_{peft_config.task_type}"
4  config = PeftConfig.from_pretrained(peft_model_id)
5  # 加载基础模型
6  model = AutoModelForCausalLM.from_pretrained(config.base_model_name_or_path)
7  # 加载PEFT模型
8  model = PeftModel.from_pretrained(model, peft_model_id)
9
10 # tokenizer编码
11 inputs = tokenizer(f'{text_column} : {dataset["test"][i]["Tweet text"]} Label
    : ', return_tensors="pt")
12
13 # 模型推理
14 outputs = model.generate(
15     input_ids=inputs["input_ids"],
16     attention_mask=inputs["attention_mask"],
17     max_new_tokens=10,
18     eos_token_id=3
19 )
20
```

```
21 # tokenizer解码
22 print(tokenizer.batch_decode(outputs.detach().cpu().numpy(),
    skip_special_tokens=True))
23
24 # 将LoRA权重合并到基础模型权重中
25 model = model.merge_and_unload()
26
27 # (可选) 保存合并后的完整模型
28 model.save_pretrained("merged_model")
29 tokenizer.save_pretrained("merged_model")
30
```